

Guía de integración con CENTINELA+

Cómo funciona el sistema real y cómo preparar vuestro trabajo para ser reutilizable

Versión 4 — Mayo 2026 · Incorpora arquitectura Medallion y actualización de grupos

Proyecto Final · Curso de Especialización en IA y Big Data · IES Dr. Lluís Simarro · Mayo 2026

Por qué leer este documento

El objetivo del proyecto no es solo que los modelos funcionen: es que **el código que escribáis pueda usarse el día que CENTINELA+ tenga datos reales del IES Simarro**. Para eso necesitáis entender cómo está organizado el sistema que está funcionando en producción, qué estructura tienen sus datos y cómo diseñar vuestro trabajo para que la transición a datos reales sea un cambio de credenciales, no una reescritura completa.

El proyecto se organiza siguiendo una **arquitectura Medallion** (bronce → plata → oro), patrón estándar en ingeniería de datos que organiza los datos en capas sucesivas de refinamiento. Usad este vocabulario en vuestra documentación:

- **Capa Bronce:** los datasets públicos en su formato original (CSV, NetCDF, ZIP), versionados en lakeFS. Son la materia prima — nunca se modifican.
 - **Capa Plata:** el InfluxDB local de cada equipo con el schema canónico de CAPTIA. Es el resultado del ETL que cada grupo construye desde su dataset de bronce.
 - **Capa Oro:** los artefactos específicos de cada caso de uso derivados de la capa plata: features para ML, embeddings para RAG, datasets etiquetados, indicadores de calidad.
-

Parte 1 — Cómo funciona CENTINELA+ por dentro

1.1 La arquitectura en cinco capas

```
[Sensores del aula]
  | MQTT (cada 5 segundos)
  v
[Mosquitto – broker MQTT]
  |
  v
[Telegraf – agente de ingesta]
  -> Extrae 5 tags del topic MQTT
  -> Routing: continuo -> telemetry
          on-change -> state_events
  |
  v
[InfluxDB 2.7 – 9 buckets]
```

```
-> telemetry (14 dias, raw)
-> telemetry_1m / _15m / _1h (rollups)
-> state_events (90 dias, senales on/off)
-> captia_metadata (inf, catalogo de variables)
    |
    v
```

[Dashboard Adapter – API REST]

```
-> Elige el bucket correcto segun el rango de tiempo consultado
-> Cachea en Redis
-> Expone /v1/query para dashboards y agentes externos
    |
    v
```

[Grafana / captia.ai / vuestros modelos]

Los sensores del aula — capa 1

En AULA01 hay dos tipos de dispositivos:

Gateway BMS (Building Management System): el dispositivo principal de control. Publica cada 5 segundos: temperatura interior (temperature_01), humedad relativa (relative-humidity), CO2 en ppm (co2), Compuestos Organicos Volatiles (t-voc), indice IAQ (iaq-index), nivel de ruido (avg-sound-level, max-sound-level), luminosidad en lux (luminosity), presencia/ocupacion (occupancy, people-count), consumo electrico (power_01), estado y control de climatizacion (ac_state, ac_control, fan_speed_01/02/03, light_01/02, valve_control, valve_state).

Sensor Sensup (modelo 0004742C0169): sensor ambiental redundante que mide 9 variables (ruido, IAQ, luminosidad, ocupacion, conteo de personas, humedad, temperatura, TVOCs).

Como sabe el sensor adonde enviar los datos?

El firmware de cada sensor viene preconfigurado por Captia con la IP del broker Mosquitto del IES Simarro y el patron de topic MQTT. Esta configuracion se realiza durante el onboarding del dispositivo mediante una interfaz web del sensor, un fichero de configuracion flasheado en el firmware, o un perfil YAML del normalizer de Captia (para dispositivos que no hablan el protocolo CAPTIA nativo).

El contrato de publicacion es minimo:

```
Topic:   captia/{env}/{tenant}/{site}/{device}/telemetry/{variable}
Payload: {"value": <numero>, "ts_ns": <timestamp en nanosegundos>}
```

El sensor no necesita saber nada mas. No habla con InfluxDB, no conoce el schema, no sabe que bucket recibira sus datos.

Mosquitto — broker MQTT — capa 2

MQTT es un protocolo de mensajeria ligero publicador/suscriptor. El sensor publica un mensaje en un "topic" (un buzzen con nombre) y cualquier servicio suscrito recibe el mensaje automaticamente.

Donde esta instalado Mosquitto en CENTINELA+? Mosquitto se ejecuta como contenedor Docker en el **servidor edge de Captia instalado fisicamente en el IES Simarro** (100.102.212.105). Es parte del stack Docker Compose de 29 servicios que Captia ha desplegado en el centro. Los sensores estan en la misma red local (OT LAN) que ese servidor, por lo que se comunican por TCP en el puerto 1883 sin salir a internet.

Como se configura Mosquitto? La configuracion vive en /etc/mosquitto/mosquitto.conf del contenedor: puerto de escucha (1883 TCP, 8883 TLS), ACLs por tenant/topic, y politicas de autenticacion. Captia provisiona las ACLs durante el onboarding de cada dispositivo.

Como se garantiza que no se pierde ningun mensaje? Captia configura QoS 1 (al menos una entrega): el broker confirma la recepcion; si no llega la confirmacion, el sensor reenvia el mensaje. Adicionalmente, Telegraf escribe en paralelo en un fichero local de durabilidad que permite reimportar datos si InfluxDB no estaba disponible temporalmente.

Telegraf — agente de ingesta — capa 3

Que es Telegraf? Telegraf es un agente open-source de recoleccion de metricas de InfluxData. Se configura con un fichero TOML declarativo y puede leer de cientos de fuentes y escribir en cientos de destinos.

Donde esta instalado Telegraf en CENTINELA+? Telegraf se ejecuta como contenedor Docker en el mismo servidor edge del IES Simarro, en el mismo Docker Compose que Mosquitto e InfluxDB. Al estar en la misma red Docker interna, se comunica con ambos sin salir a internet.

Como se configura Telegraf para hacer de puente entre Mosquitto e InfluxDB?

toml

```
# 1. INPUT: suscribirse a Mosquitto
[[inputs.mqtt_consumer]]
    servers = ["tcp://mosquitto:1883"]
    topics = ["captia/+/+/+/+/telemetry/+"]
    name_override = "captia_point"
    data_format = "json"

# 2. PROCESSOR: extraer los 5 tags del topic con regex
[[processors.regex]]
    [[processors.regex.tags]]
        key = "topic"
        pattern = "captia/([^\s]+)/([^\s]+)/([^\s]+)/([^\s]+)/[^\s]+/([^\s]+)"
        replacement = "${1}"
        result_key = "captia_env"
    # ... (idem para domain_id, site_id, asset_id, variable)

# 3. OUTPUT: escribir en InfluxDB
[[outputs.influxdb_v2]]
    urls = ["http://influxdb:8086"]
    token = "$INFLUXDB_TOKEN"
    organization = "captia"
    bucket = "telemetry"
    fieldpass = ["value"] # SOLO el campo value - descarta todo lo demas
```

El routing on-change: Telegraf tiene un segundo output para senales discretas. Usa `processors.clone` para duplicar los puntos cuya variable tiene sufijo `on-change` (`_state`, `_cmd`, `_sp`...) y `processors.dedup` (intervalo 168h) para filtrar duplicados. Solo los puntos con valor diferente al ultimo almacenado llegan al bucket `state_events`.

Quien se encarga de insertar en InfluxDB? Siempre es Telegraf, nunca el sensor directamente. El sensor solo habla MQTT; InfluxDB solo recibe de Telegraf. Esta separacion permite cambiar cualquiera de los dos componentes sin tocar el otro.

InfluxDB — base de datos de series temporales — capa 4

InfluxDB esta optimizado para series temporales: secuencias de valores numericos ordenados por tiempo. No es una base de datos relacional. La estructura interna se detalla en las secciones 1.2 y 1.3.

Dashboard Adapter y Grafana — capa 5

El Dashboard Adapter decide de que bucket leer segun el rango temporal: ultima hora -> `telemetry (raw)`, ultimas 24h -> `telemetry_1m`, ultimos 7 dias -> `telemetry_15m`, ultimo año -> `telemetry_1h`. Grafana se conecta para mostrar dashboards de monitorizacion.

1.2 El schema canonico

Que es `captia_point`?

captia_point es el nombre del measurement en InfluxDB (equivalente a una tabla en SQL). Toda la telemetria de CENTINELA+ vive en un unico measurement. Esta decision mantiene la cardinalidad baja: en lugar de tener una tabla por variable, hay una sola con un tag variable que identifica que se mide.

Un unico field: value

Cada punto tiene un field value (numero en coma flotante). La variable de la que habla se identifica por el tag variable, no por el nombre del field. Estados booleanos: 1.0 (activo) y 0.0 (inactivo).

Los 5 tags indexados

Tag	Que identifica	Ejemplo (IES Simarro)
captia_env	Entorno	prod
domain_id	Sistema	bms_classrooms
site_id	Centro/edificio	ies_simarro
asset_id	Dispositivo/aula	AULA01
variable	Magnitud medida	co2, temperature_01

El flujo completo de un dato — del sensor a InfluxDB

Escenario: el sensor de CO2 de AULA01 mide 712 ppm.

Paso 1 — El sensor publica en Mosquitto:

```
Topic:    captia/prod/bms_classrooms/ies_simarro/AULA01/telemetry/co2
Payload:  {"value": 712, "ts_ns": 1714572345000000000}
```

ts_ns es el timestamp en nanosegundos desde el epoch Unix (1 enero 1970).

Paso 2 — Telegraf extrae los 5 tags y clasifica la senal: co2 no tiene sufijo on-change => senal continua => bucket telemetry.

Paso 3 — Telegraf escribe en InfluxDB en line protocol:

```
captia_point,captia_env=prod,domain_id=bms_classrooms,site_id=ies_simarro,asset_id=A
value=712 1714572345000000000
```

Paso 4 — Flux task de downsampling (cada minuto):

```
captia_point,...,variable=co2,stat=mean  value=708.5  [timestamp_1m]
captia_point,...,variable=co2,stat=min    value=695.0  [timestamp_1m]
captia_point,...,variable=co2,stat=max    value=721.0  [timestamp_1m]
```

1.3 Los 9 buckets y que se puede ver en cada uno

telemetry — ingesta cruda (14 días)

Resolucion de 5 segundos. Para alertas en tiempo real.

```
python

query = '''
from(bucket: "telemetry")
  |> range(start: -30m)
  |> filter(fn: (r) => r.asset_id == "AULA01" and r.variable == "co2")
'''
```

Resultado: una lectura cada 5 segundos durante los ultimos 30 minutos.

telemetry_1m — granularidad 1 minuto (30 dias)

Stats: mean, min, max (analog_gauge); duty, count_rise, last (bool_presence); sum (counter).

```
python

query = '''
from(bucket: "telemetry_1m")
  |> range(start: -48h)
  |> filter(fn: (r) => r.asset_id == "AULA01"
                    and r.variable == "temperature-indoor"
                    and r.stat == "mean")
'''
```

Resultado: curva de temperatura con granularidad de 1 minuto en 48 horas.

telemetry_15m — granularidad 15 minutos (90 días)

```
python

query = '''
from(bucket: "telemetry_15m")
  |> range(start: -60d)
  |> filter(fn: (r) => r.asset_id == "AULA01"
                    and r.variable == "power_01"
                    and r.stat == "sum")
'''
```

Resultado: consumo electrico en bloques de 15 minutos durante 2 meses.

telemetry_1h — granularidad horaria (365 dias) <- el mas util para ML

Este es el bucket principal para entrenar modelos. Tiene 1 año de historia.

```
python
```

```
query = '''
from(bucket: "telemetry_1h")
  |> range(start: -365d)
  |> filter(fn: (r) => r.asset_id == "AULA01")
  |> filter(fn: (r) => r.variable == "co2" or r.variable == "temperature-indoor")
  |> filter(fn: (r) => r.stat == "mean")
  |> pivot(rowKey:["_time"], columnKey:["variable"], valueColumn:"_value")
'''
```

Resultado: DataFrame con columnas `_time`, `co2` y `temperature-indoor` listo para ML.

state_events — transiciones on-change (90 días)

Como llega un dato on-change a state_events?

Cuando el climatizador de AULA01 se enciende:

```
Topic:   captia/prod/bms_classrooms/ies_simarro/AULA01/telemetry/ac_state
Payload: {"value": 1, "ts_ns": 1714572345000000000}
```

Telegraf detecta que `ac_state` termina en `_state` (sufijo on-change). Comprueba si el valor ha cambiado: si es diferente al ultimo almacenado, escribe en `state_events`. Si no ha cambiado, descarta. El heartbeat semanal garantiza al menos un punto por semana aunque no haya cambios.

```
python
```

```
query = '''
from(bucket: "state_events")
  |> range(start: -7d)
  |> filter(fn: (r) => r.asset_id == "AULA01" and r.variable == "ac_state")
'''
```

Resultado: solo los momentos en que el climatizador cambio de estado.

captia_metadata — catalogo de variables (retencion infinita)

Dirige las Flux tasks de downsampling. Si una variable no esta aqui, no habra datos en `telemetry_1m`.

Campo	Ejemplo (co2)	Para que sirve
metric_kind	analog_gauge	Dirige downsampling y bucket destino
unit	ppm	Etiquetas en dashboards
display_name	CO2 Concentration	Nombre legible
range_min / range_max	300 / 5000	Validacion de calidad
data_type	float	Tipo del field value
is_actuator	false	Si acepta comandos

1.4 metric_kind: la clave que dirige todo

metric_kind	Tipo	Stats	Bucket
analog_gauge	Continua (T, CO2, luz)	mean, min, max	telemetry
bool_presence	Continua (ocupacion)	duty, count_rise, last	telemetry
counter	Continua (energia)	sum (delta)	telemetry
bool_state	On-change (estado AC)	last, count_rise	state_events
setpoint_step	On-change (setpoints)	last	state_events
skip	Continua sin rollup	—	telemetry

Parte 2 — La estrategia de ingesta

2.1 El principio fundamental

El objetivo no es analizar los datasets públicos. El objetivo es construir la capa plata que CENTINELA+ necesitará con datos reales.

En términos de arquitectura Medallion: vuestro dataset público es la **capa bronce**; el InfluxDB local con schema CAPTIA es la **capa plata** (el ETL que la genera es vuestra contribución evaluable); los features para ML, los embeddings para RAG o los indicadores de calidad son la **capa oro**.

Enfoque A — no reutilizable	Enfoque B — reutilizable (el correcto)
<code>df = pd.read_csv('ingauge.csv')</code>	<code>df = query_api.query_data_frame(flux_query)</code>
Modelo lee del CSV (bronce)	Modelo lee de InfluxDB (plata) con schema CAPTIA
Cuando lleguen datos reales -> reescribir todo	Cuando lleguen datos reales -> cambiar URL y token
El ETL bronce->plata no es visible ni evaluable	El ETL es visible, documentado y evaluado

Nota — datos sinteticos de CAPTIA (pendiente): estamos explorando con CAPTIA la posibilidad de que proporcionen datos sinteticos o anonimizados de una instalacion real, ya en formato InfluxDB con el schema canonico. Si esto se confirma, la capa plata de arranque ya estaria poblada y cada equipo podria construir directamente su capa oro. Os comunicaremos la respuesta de CAPTIA en cuanto la tengamos. Mientras tanto, planificad vuestro trabajo asumiendo que la capa plata la construís vosotros con ETL desde los datasets publicos.

2.2 La arquitectura de ingesta

```

Dataset publico (CSV)
|
v ETL en Python (notebook de ingesta)
|  1. Leer CSV con pandas
|  2. Normalizar: renombrar columnas, convertir unidades al SI
|  3. Asignar los 5 tags CAPTIA
|  4. Clasificar senales: continua -> telemetry / on-change ->
state_events
|  5. Convertir timestamps a nanosegundos epoch
|  6. Poblar captia_metadata con las variables del dataset
v
CAPA PLATA: InfluxDB local (mismo schema que simarro-prod)
|
v Flux tasks de downsampling
v
telemetry_1m -> telemetry_15m -> telemetry_1h
|
v
CAPA ORO: features ML / embeddings RAG / datasets etiquetados
(construida por cada equipo para su caso de uso concreto)

```

2.3 El Docker de arranque: lo que os dan los profesores y lo que debeis hacer vosotros

Los profesores os proporcionan tres ficheros listos para usar:

- `docker-compose.yml` con InfluxDB y Grafana configurados. El bootstrap de los 9 buckets con las retenciones correctas se ejecuta automaticamente al primer arranque (`docker compose up`). No hace falta ningun paso adicional de configuracion.

- `.env.example` con las variables de entorno de la conexión local (URL, token, org). Copiadlo a `.env` y nunca lo subais a Git.
- `connection.py` — plantilla de conexión a InfluxDB con variables de entorno, lista para importar en vuestros notebooks.

El Docker NO incluye datos. La ingesta de datos es responsabilidad de cada equipo. Cada grupo carga en su instancia local los datos que necesita para su caso de uso. Esto garantiza que ningún equipo está bloqueado esperando a otro.

Nota — opción adicional en negociación: estamos valorando con CAPTIA la posibilidad de que nos proporcionen un dump de InfluxDB con datos sintéticos o anonimizados de una instalación real, restaurable directamente en vuestro Docker local. Si esto se confirma, cambiaría vuestro punto de partida: en lugar de construir la capa plata desde cero con un ETL, restauraríais el dump y construiríais directamente la capa oro para vuestro caso de uso. Os comunicaremos la decisión en cuanto tengamos confirmación de CAPTIA.

El código de conexión usa variables de entorno — nunca hardcodear:

```
python

import os
from influxdb_client import InfluxDBClient

client = InfluxDBClient(
    url=os.environ.get("INFLUXDB_URL", "http://localhost:8086"),
    token=os.environ.get("INFLUXDB_TOKEN", "simarro-dev-token-2026"),
    org=os.environ.get("INFLUXDB_ORG", "captia")
)
```

```
# .env (nunca subir a Git)
INFLUXDB_URL=http://localhost:8086
INFLUXDB_TOKEN=simarro-dev-token-2026
INFLUXDB_ORG=captia
```

Cuando la infraestructura compartida esté disponible (ITI o Simarro), solo actualizáis el `.env`. El código no cambia.

Parte 3 — Adaptación de ingesta por caso de uso

Nota sobre asignaciones: la asignación del Caso A está pendiente de confirmación. El grupo G2 está valorando cambiar al Caso G, lo que dejaría el Caso A sin grupo asignado. Los profesores comunicarán la decisión en el seguimiento del próximo viernes. El resto de asignaciones son definitivas: G1 (B+H), G3 (C+E), G4 (F+D+nuevo), G5/Jorge (J), G2 mantiene el Caso I en cualquier caso.

Caso A — Ingeniería de Datos y Pipeline de Ingesta

Asignacion pendiente de confirmacion. El equipo que inicialmente tenia este caso (G2) esta valorando cambiar al Caso G. Los profesores comunicaran la decision final en el seguimiento del viernes. Si el Caso A queda sin grupo asignado, los profesores proporcionaran el codigo de referencia del pipeline como documentacion del proyecto.

El objetivo no es solo cargar datos. Es simular exactamente como CENTINELA+ recibe datos de sensores reales.

Este es el unico caso de uso que debe reproducir el flujo completo IoT: sensor -> MQTT -> Telegraf -> InfluxDB. Los demas equipos pueden insertar directamente en InfluxDB desde Python; el equipo A debe simular el proceso completo.

Sobre la velocidad de simulacion: Los datasets tienen resolucion de minutos o segundos. Publicar a la misma velocidad que el dataset original seria extremadamente lento. La simulacion debe ir a velocidad acelerada: publicar cada fila con el timestamp original del dataset en ts_ns, pero sin esperar entre filas el tiempo real transcurrido.

```
python

import paho.mqtt.client as mqtt
import pandas as pd, json

df = pd.read_csv('ingauge_aula01.csv')
client = mqtt.Client()
client.connect("localhost", 1883)

for _, row in df.iterrows():
    ts_ns = int(pd.Timestamp(row['timestamp']).timestamp() * 1e9)
    payload = json.dumps({"value": float(row['co2']), "ts_ns": ts_ns})
    client.publish(
        "captia/prod/bms_classrooms/ies_simarro/AULA01/telemetry/co2",
        payload
    )
    # Sin sleep - publicar a la maxima velocidad posible
```

Que bucket se rellena segun el tipo de variable:

Tipo	Ejemplo	Proceso	Bucket
Continua (sensor analogico)	CO2, temperatura	Publish en topic telemetry/{variable}. Telegraf escribe.	telemetry -> rollups
Discreta con transiciones	Estado climatizador	Publish en topic telemetry/{variable_state}. Telegraf detecta cambio.	state_events
Contador acumulado	Consumo electrico Wh	metric_kind=counter en metadata.	telemetry -> rollup sum

No se deben inventar datos. Si el dataset no tiene una variable, simplemente no se ingesta.

Mapping In-Gauge/En-Gage -> CAPTIA:

Variable CSV	variable CAPTIA	metric_kind	Bucket
Indoor_CO2	co2	analog_gauge	telemetry
Indoor_Temp	temperature-indoor	analog_gauge	telemetry
Indoor_Hum	relative-humidity	analog_gauge	telemetry
Indoor_Noise	avg-sound-level	analog_gauge	telemetry
Occupied	occupancy	bool_presence	telemetry
CoolingState	ac_state	bool_state	state_events
HeatingState	heat_state	bool_state	state_events
Outdoor_Temp	temperature_outdoor	analog_gauge	telemetry
Outdoor_Solar	solar_irradiance	analog_gauge	telemetry

Tags: captia_env=prod, domain_id=bms_classrooms, site_id=ies_simarro, asset_id=AULA01..AULA16

Caso B — Prediccion de Consumo Electrico (G1: Sergio, Ainhoa, Guillermo, Jordi)

El objetivo no es solo un modelo de forecasting. Es un modelo cuyo codigo de extraccion de datos sea reutilizable directamente con CENTINELA+.

Ingesta de BDG2: seleccionad 5-10 edificios educativos durante 1 año (no los 53M registros completos).

Tags: domain_id=bms_buildings, site_id=bdg2_education, asset_id=bdg2_bldg_XXXX, variable=power_01

Mapping UCI Appliances -> CAPTIA:

Variable	variable CAPTIA	metric_kind
Appliances (Wh)	power_01	counter
lights (Wh)	light_consumption	counter
T1-T9 (°C)	temperature_01-_09	analog_gauge
RH_1-RH_9 (%)	relative-humidity_01-_09	analog_gauge
T_out (°C)	temperature_outdoor	analog_gauge

Caso C — Deteccion de Anomalias en HVAC (G3: Joan Juan, Edgar, Ivan, Joan Benavent)

Mapping LBNL FDD -> CAPTIA:

Variable LBNL FDD	variable CAPTIA	metric_kind	Bucket
SA_TEMP	temperature_supply	analog_gauge	telemetry
RA_TEMP	temperature_return	analog_gauge	telemetry
OA_TEMP	temperature_outdoor	analog_gauge	telemetry
CCV	valve_control	setpoint_step	state_events
FAN_STATE	fan_speed_01_state	bool_state	state_events
SA_CFM	fan_speed_01	analog_gauge	telemetry
OCCU_MOD	occupancy	bool_presence	telemetry

Las etiquetas de fallo no van en InfluxDB: van en lakeFS o en un measurement separado `captia_fault_labels`.

Tags: `domain_id=hvac_system, site_id=lbnl_building59, asset_id=RTU_01`

Caso D — Calidad del Aire, Confort Interior y Ocupacion (G4)

El mapping mas directo a CENTINELA+. Coordinarse con G3 (caso E) para reutilizar las variables exteriores.

```
python

query = '''
from(bucket: "telemetry_1m")
  |> range(start: -30d)
  |> filter(fn: (r) => r.asset_id == "AULA01")
  |> filter(fn: (r) => r.variable == "co2" or r.variable == "temperature-indoor"
                    or r.variable == "luminosity" or r.variable == "avg-sound-level")
  |> filter(fn: (r) => r.stat == "mean")
  |> pivot(rowKey:["_time"], columnKey:["variable"], valueColumn:"_value")
'''
```

Caso E — Datos Meteorologicos (G3)

Mapping ERA5 -> CAPTIA:

Tags: captia_env=prod, domain_id=weather_station, site_id=xativa, asset_id=era5_gridpoint

Variable ERA5	variable CAPTIA	Conversion	metric_kind
2m_temperature (K)	temperature_outdoor	K - 273.15	analog_gauge
surface_solar_radiation_downwards (J/m2)	solar_irradiance	/ 3600 = W/m2	analog_gauge
total_precipitation (m)	precipitation	x 1000 = mm	counter
velocidad viento	wind_speed	sqrt(u2+v2) m/s	analog_gauge
surface_pressure (Pa)	pressure	/ 100 = hPa	analog_gauge

Caso F — MLOps (G4)

El equipo F no genera ingesta propia. Su aporte transversal:

- Definir y comunicar la convencion de nomenclatura de experimentos de MLflow antes del viernes de la semana 1.
- Versionar en lakeFS el schema de tags y variables CAPTIA.
- Garantizar que todos los experimentos de MLflow referencian el tag de lakeFS del dataset.

Caso G — Calidad de Datos con Agentes Especialistas (G2: Oscar, Vicent, David — pendiente de confirmacion)

Nota: El grupo G2 esta valorando cambiar del Caso A a este Caso G. La decision se confirmara en el seguimiento del viernes. Si el cambio se confirma, G2 llevara el Caso G junto al Caso I.

Posicion en el medallion y estrategia anti-bloqueo

El equipo G tiene una posicion transversal unica: **audita la calidad de todas las capas** (bronce, plata y oro) de todos los equipos. La clave para no quedarse bloqueado esperando datos de otros es trabajar en **oleadas progresivas**, publicando reglas antes de que los demas equipos hayan terminado:

Semana 1 — Reglas sobre la capa bronce (sin depender de nadie): Podeis definir y ejecutar reglas sobre los datasets publicos originales sin esperar a ningun otro equipo. Ejemplos con Great Expectations:

```
python
```

```
# Reglas sobre capa bronce (CSV original, no necesita InfluxDB)
validator.expect_column_values_to_be_between("Indoor_CO2", min_value=300, max_value=
validator.expect_column_to_exist("timestamp")
validator.expect_column_values_to_not_be_null("Indoor_Temp")
validator.expect_column_values_to_be_of_type("Occupied", type_="int")
```

Semana 2 — Reglas sobre la capa plata (cuando los demas tengan InfluxDB cargado):

```
python

# Reglas sobre capa plata en InfluxDB
# Completitud: registros esperados vs. reales
# Si dataset tiene 30 dias a 1min -> 43.200 puntos esperados por variable

# Consistencia: valores fuera de rango fisico
query_outliers = '''
from(bucket: "telemetry_1m")
  |> range(start: -30d)
  |> filter(fn: (r) => r.variable == "co2")
  |> filter(fn: (r) => r._value < 300 or r._value > 5000)
  |> count()
'''

# Tambien verificar: los 5 tags presentes, field value numerico,
# senales on-change en state_events (no en telemetry),
# captia_metadata poblado para todas las variables.
```

Issues reales de simarro-prod que son material de trabajo directo:

- H-1: site_id inconsistente entre buckets (ies_simarro vs ies_carlos_iii)
- H-2: registry.yaml usa nombre diferente al de los datos
- H-3: datos de entorno dev mezclados con produccion
- Issue #27: override asset_id del normalizer solo aplica a metadata
- Issue #29: --retention 0 en bucket create aplica 720h por defecto, no infinita

Semana 3 — Reglas sobre la capa oro + agentes evaluadores:

- Balance de clases en datasets de entrenamiento.
- Experimentos MLflow con baseline documentada y referencia al tag lakeFS.
- Agente evaluador del chatbot: llama al endpoint de G1 con el golden set y mide relevancia, coherencia y tasa de alucinacion.

Agentes especializados de calidad

```
python
```

```
# Agente 1: Validador de capa plata
@tool
def validate_silver_layer(group: str, asset_id: str) -> str:
    """Ejecuta reglas sobre schema, completitud, rangos y consistencia temporal."""

# Agente 2: Evaluador del chatbot (coordinar con G1 semana 2)
@tool
def evaluate_chatbot_response(question: str, expected_answer: str) -> dict:
    """Llama al endpoint del chatbot y evalua relevancia, coherencia y alucinacion."""

# Agente 3: Auditor de experimentos MLflow (coordinar con G4-F)
@tool
def audit_mlflow_experiment(experiment_name: str) -> dict:
    """Verifica baseline, metricas y referencia al tag lakeFS del dataset."""
```

Regla de oro del equipo G: publicad las reglas antes de que los demas terminen. Si las reglas llegan al final del proyecto, nadie puede corregir nada. Cada semana que publicais reglas es una semana que los demas tienen para mejorar.

Caso H — Sistema RAG, Agentes de IA y Chatbot (G1: Sergio, Ainhoa, Guillermo, Jordi)

Preguntas del equipo H

Hay que esperar a que el equipo E (G3) termine para tener los datos ERA5?

No. Podeis trabajar en paralelo desde el primer dia:

- Podeis descargar ERA5 directamente vosotros — no necesitais esperar a G3. Si quereis coordinaros para no duplicar la descarga (puede tardar horas para rangos largos), es una optimizacion opcional, no un requisito.
- Lo que si necesitareis de G3 son los modelos que entrenen (prediccion meteorologica, deteccion de anomalias HVAC). Pero eso es para la semana 3, no para la semana 1.
- **Y lo mas importante:** no necesitais que los modelos de G3 esten listos para construir las tools. Podeis mockearlos desde el primer dia (ver abajo).

Es necesario un EDA de los datos ERA5?

No es prioritario para vuestro caso de uso. El foco del Caso H no es analizar los datos meteorologicos — eso es trabajo de G3. Vuestro foco es **la arquitectura del sistema de agentes y tools**. Podeis cargar ERA5 en InfluxDB con el ETL minimo necesario para que las queries Flux funcionen, y pasar directamente a construir el chatbot.

La estrategia clave del Caso H: mockear para avanzar sin esperar

El foco del Caso H no es la veracidad de los modelos que llama. Es la capacidad de articular un sistema de agentes que sea capaz de utilizar tools que llamen a esos modelos y funciones.

Los modelos de predicción de G3 (meteorología, HVAC) y el propio modelo B (consumo) no estarán listos hasta la semana 3. Pero eso no tiene que bloquearos ni un día. Desde el primer día podeis usar **mocks**: implementaciones falsas que devuelven datos plausibles con el mismo contrato de interfaz que el modelo real.

```
python

# Semanas 1-2: mock de la tool de predicción meteorológica
# (misma firma que tendrá la versión real – solo cambia la implementación)
@tool
def get_weather_prediction(variable: str, horizon_hours: int = 24) -> str:
    """Genera predicción para una variable meteorológica."""
    # MOCK: datos plausibles hardcodeados para desarrollo
    mock_values = {
        "temperature_outdoor": f"{22.5 + horizon_hours * 0.1:.1f}°C",
        "solar_irradiance": f"{350.0:.0f} W/m²",
        "precipitation": "0.0 mm"
    }
    return f"Predicción ({horizon_hours}h): {mock_values.get(variable, 'variable no')}

# Semana 3: cuando G3 entregue su modelo, sustituir por la implementación real
@tool
def get_weather_prediction(variable: str, horizon_hours: int = 24) -> str:
    """Genera predicción para una variable meteorológica."""
    # REAL: llamada al modelo de G3
    from g3_models import weather_predictor
    return weather_predictor.predict(variable, horizon_hours)
```

El enrutador, la lógica de selección de agentes, el RAG sobre Elasticsearch y el LLM local funcionan exactamente igual con el mock que con el modelo real. Cuando G3 os entregue la función, es un cambio de una línea.

Arquitectura del chatbot: tools sobre InfluxDB + Elasticsearch contextual

El objetivo no es solo un chatbot que responda. Es un chatbot cuyas herramientas de consulta de datos funcionen directamente con CENTINELA+ cuando lleguen datos reales del IES Simarro.

La arquitectura tiene dos componentes con propósitos distintos:

1. Tools sobre InfluxDB — para datos numéricos precisos

Cuando el usuario pregunta "¿cuál fue la temperatura media en Xativa en enero de 2024?", la respuesta es un número calculado en tiempo real. Indexar esa información como texto en

ElasticSearch no garantiza precision: el fragmento puede estar desactualizado o no corresponder exactamente a la pregunta. Una query Flux devuelve el numero exacto sin ambigüedad.

2. ElasticSearch con RAG — para conocimiento general y contextual

Cuando el usuario pregunta "por que sube el CO2 en un aula?" o "que significa un nivel de IAQ de 150?", la respuesta es conocimiento general que no depende de los valores numericos almacenados en InfluxDB. Para estas preguntas, indexar documentos explicativos en ElasticSearch y usar RAG es la opcion correcta. Estos documentos los podeis preparar desde el primer dia sin datos.

Que documentos indexar en ElasticSearch:

- Explicaciones sobre IAQ, rangos recomendados de CO2, temperatura y humedad en aulas (OMS, normativa española).
- Descripcion del sistema CENTINELA+ y las variables que mide.
- Informacion climatica general sobre Valencia/Xativa.
- Resúmenes de los datasets del proyecto.

No indexeis en ElasticSearch los valores numericos de los datasets. Esos datos viven en InfluxDB y se consultan mediante tools.

Tipo de pregunta	Ejemplo	Mecanismo	Por que
Dato numerico preciso	Temperatura media enero 2024	Tool query_influxdb	Valor exacto en tiempo real
Comparacion temporal	Fue mas caluroso julio 2023 o 2022?	Tool query_influxdb x2	Dos valores precisos
Prediccion futura meteo	Que temperatura habra manana?	Tool get_weather_prediction (mock->G3)	LLM no predice — llama al modelo
Prediccion consumo	Cuanta energia consumira AULA01 manana?	Tool get_consumption_prediction (mock->G1/B)	Mock en semanas 1-2, real en semana 3
Estado actual edificio	Esta encendido el AC de AULA01?	Tool query_influxdb sobre state_events	Query transiciones recientes
Anomalias HVAC	Hay alguna averia en la climatizacion?	Tool check_hvac_anomaly (mock->G3)	Mock en semanas 1-2, real en semana 3
Conocimiento general	Que nivel de CO2 es peligroso?	RAG sobre ElasticSearch	Informacion documental
Normativa	Temperatura recomendada en aulas?	RAG sobre ElasticSearch	Informacion general indexada

Conjunto minimo de tools propuesto

python

```
# TOOL 1: Consulta generica a InfluxDB
```

```
@tool
```

```
def query_influxdb(variable: str, start: str, aggregation: str = "mean",
                   asset_id: str = "era5_gridpoint") -> str:
    """
    Consulta el valor de una variable en InfluxDB.
    variable: nombre CAPTIA (co2, temperature_outdoor, solar_irradiance...)
    start: periodo (-7d, -30d, 2024-01-01T00:00:00Z...)
    aggregation: mean, max, min, sum, last
    asset_id: AULA01 (edificio) o era5_gridpoint (meteorologia)
    Devuelve el valor calculado con la funcion de agregacion indicada.
    """
```

```
# TOOL 2: Comparacion entre dos periodos
```

```
@tool
```

```
def compare_periods(variable: str, period1_start: str, period1_end: str,
                    period2_start: str, period2_end: str,
                    aggregation: str = "mean") -> str:
    """
    Compara el valor de una variable entre dos periodos temporales.
    Util para: "Fue mas caluroso X o Y?"
    """
```

```
# TOOL 3: Prediccion meteorologica - COORDINAR CON G3 (caso E)
```

```
@tool
```

```
def get_weather_prediction(variable: str, horizon_hours: int = 24) -> str:
    """
    Genera prediccion de una variable meteorologica.
    REQUIERE integracion con el modelo ERA5 entrenado por G3 (caso E).
    Planificar coordinacion con G3 desde la semana 1.
    """
```

```
# TOOL 4: Prediccion de consumo - COORDINAR CON G1 mismo (caso B)
```

```
@tool
```

```
def get_consumption_prediction(asset_id: str = "AULA01",
                               horizon_hours: int = 24) -> str:
    """
    Genera prediccion de consumo electrico para un asset.
    Usa el modelo de prediccion entrenado en vuestro propio caso B.
    Integrarlo en la semana 3 cuando el modelo este listo.
    """
```

```
# TOOL 5: Estado actual del edificio
```

```
@tool
```

```
def get_building_state(asset_id: str = "AULA01") -> str:
    """
    Consulta el estado actual del edificio: climatizacion, iluminacion,
    ocupacion y condiciones ambientales en tiempo real.
    """
```

```
Lee de state_events (last value) y telemetry (ultima hora).
"""

# TOOL 6: Deteccion de anomalias HVAC - COORDINAR CON G3 (caso C)
@tool
def check_hvac_anomaly(asset_id: str = "AULA01") -> str:
    """
    Detecta anomalias en el sistema HVAC del asset.
    REQUIERE integracion con el modelo de deteccion entrenado por G3 (caso C).
    Planificar coordinacion con G3 desde la semana 1.
    """
```

Coordinacion prioritaria con otros grupos

Un chatbot que integra modelos predictivos de otros grupos es cualitativamente diferente a uno que solo responde preguntas historicas. Es uno de los aspectos mas valorables del proyecto.

Tool	Grupo	Cuando	Que necesitais
get_weather_prediction	G3 (caso E)	Semana 3	Endpoint o funcion Python del modelo ERA5
get_consumption_prediction	G1 (vosotros, caso B)	Semana 2-3	El modelo de forecasting que estais entrenando
check_hvac_anomaly	G3 (caso C)	Semana 3	Endpoint o funcion Python del modelo HVAC

Accion semana 1: definir con G3 la interfaz de sus modelos (que recibe y que devuelve cada funcion Python). Asi podeis escribir el stub de la tool desde el principio y G3 solo tiene que rellenar la implementacion cuando el modelo este listo.

Si conseguis que el chatbot llame al modelo de prediccion del caso B para responder "cuanta energia consumira AULA01 manana?", el resultado es un sistema de IA completo e integrado — exactamente el tipo de arquitectura que se desplegara en CENTINELA+ con datos reales.

Caso I — Big Data: Benchmark Spark vs. Pandas (G2: Oscar, Vicent, David)

No necesita cargar todo BDG2 en InfluxDB. El equipo G2 puede generar el subconjunto reducido (5-10 edificios educativos, 1 año) que G1 (caso B) cargara en InfluxDB.

Tags para el subconjunto: domain_id=bms_buildings, site_id=bdg2_education, asset_id=bdg2_bldg_XXXX, variable=power_01

Caso J — Trafico + Vision Artificial YOLOv (G5: Jorge Albert Bosch)

Prioridad absoluta semana 1: el pipeline de captura de imagenes e inferencia YOLOv debe estar operativo desde el primer dia. Cada dia sin datos acumulados es una perdida irrecuperable de serie temporal.

Capa bronce: imagenes JPEG de camaras DGT (capturas periodicas) + datos meteorologicos AEMET (JSON via API). Versionados por fecha en lakeFS.

ETL bronce -> plata:

Las imagenes NO van en InfluxDB (InfluxDB solo admite valores numericos). Almacenarlas en MinIO (S3 compatible, incluido en el stack Docker) con la ruta

`cameras/{camera_id}/{date}/{timestamp}.jpg`.

Los conteos de vehiculos detectados SI van en InfluxDB:

```
python

# Line protocol para conteo de vehiculos
captia_point,captia_env=prod,domain_id=traffic_cameras,
  site_id=valencia,asset_id=DGT_CAM_V46_001,variable=vehicle_count
  value=12 [timestamp_ns]
```

Variables en InfluxDB: `vehicle_count` (analog_gauge), `congestion_level` (0=libre, 1=fluido, 2=denso, 3=congestionado, analog_gauge), `detection_confidence` (analog_gauge).

Capa oro: serie temporal de conteos + variables meteorologicas AEMET fusionadas por timestamp -> features para el modelo de prediccion de congestion (XGBoost o Random Forest).

Nota para trabajo en remoto desde Galicia: el pipeline debe ejecutarse de forma desatendida con un cron job o APScheduler. Guardad los logs de ejecucion para detectar camaras que fallen intermitentemente.

Donde desplegar el stack Docker (LakeFS, MLflow, MinIO, Ollama, Elasticsearch, Kibana, PostgreSQL)?

La infraestructura definitiva esta pendiente de confirmar con Manuel (ITI) y Alberto Aparicio (IES Simarro). Se estan valorando: servidor GPU en el ITI (para Ollama), servidor general en el ITI (para InfluxDB, MLflow, lakeFS), servidor en el Simarro (posible para InfluxDB y Grafana).

Este detalle no debe bloquearos. El proceso inicial que podeis hacer ahora sin esperar:

1. Desplegar el stack Docker en vuestra maquina local con docker-compose up.
2. Comenzar a trabajar con el InfluxDB local (Docker de arranque de los profesores).
3. Cuando se resuelva la infraestructura compartida, actualizar solo el .env.

Que hacer si el stack Docker no esta listo y necesito empezar? El EDA de vuestro dataset podeis hacerlo directamente del CSV desde el primer dia. pandas, matplotlib y seaborn son suficientes para el EDA. Guardad los resultados en un notebook documentado — sera la base para decidir que subconjunto ingestar en InfluxDB.

Parte 5 — Checklist antes de entrenar modelos

- ☐ InfluxDB local levantado en <http://localhost:8086>
 - ☐ Los 9 buckets creados con retenciones correctas
 - ☐ captia_metadata poblado (sin esto, los rollups no generan datos)
 - ☐ Ingesta completada: registros esperados vs. reales verificados
 - ☐ Query Flux basica desde Python devuelve un DataFrame con datos
 - ☐ Los 5 tags correctos y field value numerico
 - ☐ Senales on-change en state_events, continuas en telemetry
 - ☐ Dataset versionado en lakeFS con tag etiquetado
 - ☐ Experimento MLflow referencia el tag de lakeFS
 - ☐ Conexion a InfluxDB usa variables de entorno (.env), nunca hardcodeada
-

Parte 6 — Coordinacion entre equipos

Grupos y casos de uso

Grupo	Integrantes	Casos de uso
G1	Sergio, Ainhoa, Guillermo, Jordi	B (Prediccion consumo) + H (RAG/Chatbot)
G2	Oscar, Vicent Benavent, David Pallet	I (Big Data benchmark) + G (Calidad con agentes, en evaluacion — puede mantenerse A)
G3	Joan Juan, Edgar Tormo, Ivan Tormo, Joan Benavent	C (Anomalias HVAC) + E (Meteorologia)
G4	Maria Galbis, MJ Garcia, Federico Brond, Lucia Fasanar, Jose Vento	F (MLOps) + D (Calidad aire) + Nuevo (Test calidad agentes)
G5	Jorge Albert Bosch	J (Trafico + vision artificial YOLOv)

- Nota sobre G2:** el equipo G2 esta valorando cambiar el Caso A (Pipeline IoT) por el Caso G (Calidad con agentes). El Caso I (Big Data benchmark) se mantiene en ambas opciones. La decision esta pendiente porque el Caso G requiere una coordinacion muy activa con el resto de equipos para no quedarse bloqueado (ver seccion de coordinacion G2). Los profesores comunicaran la decision en la proxima sesion de seguimiento.
- Nota sobre Caso A sin asignar:** si G2 confirma el cambio a G, el Caso A queda sin grupo. Las opciones que estan valorando los profesores: (1) que cualquier equipo lo incorpore como extension opcional; (2) que los profesores proporcionen el pipeline del Caso A como codigo de referencia documentado para el proyecto sin entregable formal.

Mapa de dependencias

G4-F (MLOps) -> convencion experimentos MLflow -----> TODOS (semana 1-2)
G2-G (Calidad, si confirma cambio) -> reglas bronce -----> TODOS (semana 1-2)

G2 (subconjunto BDG2) -> edificios educativos -----> G1 (Caso B)
(coordinar: G2 selecciona en benchmark Spark, G1 carga en InfluxDB)

G1 (Caso B) -> modelo prediccion consumo -----> G1 (Caso H, tool propia)

G3 (Caso E) -> modelo prediccion meteorologica -----> G1 (Caso H, tool get_weather_prediction)

G3 (Caso E) -> datos ERA5 en InfluxDB -----> G1 (Caso H, datos historicos)
(coordinar para no duplicar descarga)

G3 (Caso C) -> modelo deteccion anomalias HVAC -----> G1 (Caso H, tool check_hvac_anomaly)

G4 (Caso D) -> datos IAQ/ocupacion en InfluxDB -----> G1 (Caso H, agente de edificio)

G4 (Caso F) -> MLflow + lakeFS -----> todos los grupos
(infraestructura MLOps compartida)

G4 (Nuevo) -> test calidad con agentes -----> G1 (Caso H)
(agentes que evaluan respuestas del chatbot)

Sugerencias de coordinacion por pares de grupos

G1 <-> G2

G1 necesita el subconjunto de BDG2 que selecciona G2 (caso I) como fuente principal para el caso B. Coordinar antes del viernes semana 1: que edificios educativos, que tags, que bucket. Mientras tanto, G1 puede empezar con UCI Appliances Energy.

Si G2 confirma el cambio a G (Calidad): el equipo G2 auditara la calidad de los datos y los modelos de todos los grupos. Para no bloquearse, G2 debe publicar reglas de calidad por oleadas: reglas sobre bronce en semana 1 (sobre los CSV originales, sin necesitar a nadie), reglas sobre plata en semana 2 (cuando los demas equipos tengan InfluxDB cargado), reglas sobre oro en semana 3.

G1 <-> G3

Las dos tools mas importantes del chatbot dependen de modelos de G3 (prediccion meteorologica del caso E, deteccion anomalias del caso C). Esta coordinacion debe planificarse desde la semana 1 aunque la integracion llegue en la semana 3.

Accion semana 1: G1 y G3 definen juntos la interfaz de los modelos (firma de la funcion Python: que recibe, que devuelve). G1 escribe el stub de la tool desde el principio; G3 solo tiene que implementar la funcion cuando el modelo este listo.

G1 y G3 pueden coordinar la descarga de ERA5 — ambos la necesitan. Si G3 la descarga primero, G1 puede reutilizarla directamente.

G3 <-> G4

Los casos C y D trabajan con variables de confort interior y HVAC que son complementarias. Si G3 ingesta variables HVAC y G4 ingesta variables ambientales con los mismos tags (mismo site_id, mismo asset_id), el cruce de ambos datasets por timestamp enriquece ambos analisis. Coordinar los tags en la semana 1.

G4 (F — MLOps) es el equipo de soporte de todos. Definir y comunicar la convencion de experimentos de MLflow antes del viernes semana 1.

G4 (Nuevo caso) <-> G1

El caso nuevo de G4 — test de calidad con agentes especialistas — tiene sinergia natural con el chatbot de G1. Un agente evaluador que audita las respuestas del chatbot cierra el ciclo de calidad del proyecto.

Sugerencia: G4 diseña el golden set de preguntas de evaluacion en la semana 2. G1 expone el chatbot con un endpoint /ask?q=... G4 diseña agentes que llaman a ese endpoint, comparan la respuesta con la esperada y generan un informe automatizado de calidad.

G5 (Jorge) — trabajo autonomo desde Galicia

Jorge trabaja en solitario y en remoto. Su pipeline de captura de imagenes + inferencia YOLOv debe estar operativo desde el primer dia. Las imagenes van a MinIO (no a InfluxDB); los conteos de vehiculos van a InfluxDB con dominio traffic_cameras. Su caso no tiene dependencias criticas con otros grupos, lo que facilita el trabajo autonomo.